APPLIED AI SERIES — Tokalator.wiki

Model Context Protocol (MCP)

Servers · Clients · Tools · Resources · Prompts

Transport · Sampling · Scaling

Vahid Farajijobehdar

Sr. Applied AI Specialist

Agenda — From MCP Basics to Production Patterns

What we cover today, in order

1. **Foundations** — What MCP is, the problem it solves, core architecture
2. **Core Concepts** — Servers, clients, tools, resources, prompts, and the inspector
3. **Transport Layer** — stdio, streamable HTTP, SSE connections, config flags
4. **Advanced Topics** — Sampling, logging, progress, roots, scaling, stateless HTTP

Part 5 — Live Session (live!)

Real MCP walkthrough — AI chatbot scenario:

- Build a chatbot support server with tools
- Test with the MCP Inspector
- Connect a client end-to-end
- Add sampling and progress notifications
- Deploy with the right transport

Part 1 — Foundations

What is MCP? · The problem it solves · Core architecture.

1.1 — What is MCP?

A plug-and-play standard for giving AI assistants real capabilities

The Short Answer

MCP (Model Context Protocol) is a communication standard that lets your AI assistant *connect to real tools and data* things like your CRM, ticketing system, or internal knowledge base without you having to build every integration from scratch.

A Concrete Example

You're building a customer support chatbot. Without MCP, your team writes custom code to connect Claude to Zendesk, Salesforce, and your internal docs. With MCP, you plug in three pre-built MCP servers and you're done.

Analogy: MCP is like a USB-C standard for AI tools. Any

MCP = standardized tool integration

- No MCP → build every integration yourself
- With MCP → plug in servers that already exist
- Your chatbot stays clean; MCP handles the details
- Service providers publish official MCP servers (e.g. Atlassian, Salesforce, AWS)

Think of it as npm for AI integrations install a package instead of writing it.

1.2 — The Problem MCP Solves

Building an AI assistant without MCP is an integration nightmare

Real Scenario: Customer Support Chatbot

A user asks your chatbot: "What's the status of my ticket #4821?" For Claude to answer, it needs tools to query Zendesk. But Zendesk has hundreds of API endpoints.

Without MCP — your team writes all of this:

```
get_ticket(ticket_id)
list_tickets(user_id)
update_ticket_status(...)
add_comment(ticket_id, text)
escalate_ticket(ticket_id)
...  40+ more functions ...
```

In plain English: *Every function needs a schema Claude can read, error handling, API auth, and ongoing maintenance. That's weeks of work before you even start on the chatbot itself.*

MCP shifts who does the work

Without MCP	With MCP
Your team writes schemas	Zendesk MCP server has them
Your team calls the API	MCP server handles it
Your team maintains code	MCP maintainer does
Weeks of setup	Hours of setup

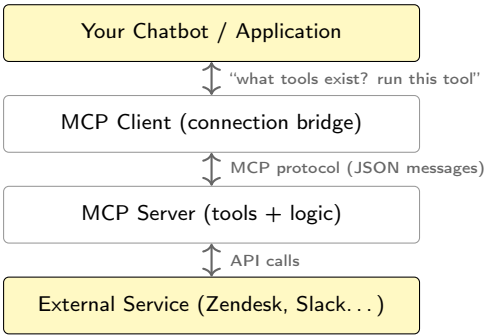
With MCP — you write one line:

```
connect_mcp_server(
"zendesk-mcp-server"
)
```

Someone still builds the tools — you just don't have to. Often the service provider does it for you.

1.3 — Core Architecture

Four layers — each with one clear job



Each layer has one job

Layer	Responsibility
Your App	Send user messages to Claude
MCP Client	Manage the server connection
MCP Server	Define tools, call the API
External	The real service (Zendesk, etc.)

In plain English: *Your chatbot doesn't talk to Zendesk directly. It talks to the MCP server, which talks to Zendesk. Clean separation — your app doesn't need to know anything about Zendesk's API.*

1.4 — End-to-End Request Flow

"What's the status of my ticket #4821?" — step by step

Step	Who	What Happens
1	User → Chatbot	User types the question
2	Chatbot → MCP Client	"What tools do we have?"
3	MCP Client → MCP Server	Sends `ListToolsRequest`
4	MCP Server → MCP Client	Returns tool list (e.g. `get_ticket`, `add_comment`)
5	Chatbot → Claude	User question + available tools
6	Claude → Chatbot	"I need to call `get_ticket` with id=4821"
7	Chatbot → MCP Client	"Run `get_ticket(4821)`"
8	MCP Client → MCP Server	Sends `CallToolRequest`
9	MCP Server → Zendesk	Makes the real Zendesk API call
10	Zendesk → Chatbot	Ticket data flows back as `CallToolResult`
11	Chatbot → Claude	"Here's the ticket data"
12	Claude → User	"Your ticket is open, assigned to support team."

Many steps — but your code only touches steps 2, 5, 7, and 11. MCP handles the rest.

Part 2 — Core Concepts

MCP Servers · Tools · Resources · Prompts · Inspector

2.1 — What an MCP Server Exposes

Three types of capability — pick the right one for each use case

Type	What It Is	Chatbot Example
Tools	Functions Claude can call (actions)	`create_ticket()`, `send_message()`
Resources	Read-only data Claude can fetch	Ticket list, FAQ articles
Prompts	Pre-built expert instructions	"Escalate this ticket" template

In plain English: *Think of tools as buttons (do something), resources as pages (read something), and prompts as macros (run a pre-written script).*

Setting up a server — just one line:

```
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("SupportBot", log_level="ERROR")
```

Rule of thumb

- Does it **change** something? → Tool
- Does it **read** something? → Resource
- Is it a **reusable workflow**? → Prompt

Why FastMCP?

- No manual JSON schema writing
- Python type hints auto-generate schemas
- Claude gets properly formatted tool specs
- Error handling works naturally

The SDK reads your function signatures and generates everything Claude needs to understand the tool.

2.2 — Tools: Actions the Chatbot Can Take

Define a function in Python — Claude can call it automatically

Example: Support ticket tool

```
@mcp.tool(
name="get_ticket_status",
description="Look up the status of a
customer support ticket."
)
def get_ticket_status(
ticket_id:  str = Field(
description="The ticket ID, e.g.  4821"
)
):
ticket = zendesk.get(ticket_id)
if not ticket:
raise ValueError("Ticket not found")
return ticket.status
```

In plain English: *`@mcp.tool` registers the function as a tool. `Field(description=...)` tells Claude what each argument means. The SDK generates the JSON schema automatically — you never write it by hand.*

What the SDK generates for Claude

```
{
"name":  "get_ticket_status",
"description":  "Look up the status
of a customer support ticket.",
"parameters":  {
"ticket_id":  {
"type":  "string",
"description":  "The ticket ID"
}
}
}
```

In plain English: *This JSON is what Claude reads to understand the tool. You never write this yourself — the SDK builds it from your Python function.*

2.3 — Resources: Read-Only Data Endpoints

Expose data the chatbot can fetch — no tool call needed

Direct resource — list all open tickets:

```
@mcp.resource(
"support://tickets/open",
mime_type="application/json"
)
def list_open_tickets() -> list[str]:
return zendesk.get_open_tickets()
```

In plain English: *A static URL — always returns the same kind of data. Like a GET endpoint: no input needed, just fetch and return.*

Templated resource — fetch one ticket:

```
@mcp.resource(
"support://tickets/{ticket_id}",
mime_type="text/plain"
)
def fetch_ticket(ticket_id:  str) -> str:
return zendesk.get_ticket(ticket_id)
```

Resources vs Tools — when to use each

	Tool	Resource
Use for	Actions, writes	Reads only
Changes data?	Yes	No
Example	Create ticket	List tickets

Key UX use case: `@mentions`

User types `@faq-refund-policy` in the chat. Your app fetches the resource and injects the content directly into the prompt — Claude reads it instantly, no tool call required.

Use `mime_type` to signal the data format: `application/json` for structured data, `text/plain` for readable text.

2.4 — Prompts: Pre-Built Expert Workflows

Package your best prompts so every client benefits automatically

Example: ticket escalation prompt

```
@mcp.prompt(
name="escalate",
description="Escalate a ticket with
a professional summary."
)
def escalate_ticket(
ticket_id:  str = Field(...)
)  -> list[base.Message]:
prompt = f"""
You are a senior support agent.
Review ticket {ticket_id}.
1.  Summarise the issue in 2 sentences.
2.  Identify urgency level (P1-P4).
3.  Draft an escalation message
to the engineering team.
Use 'get_ticket_status' tool first.
"""
return [base.UserMessage(prompt)]
```

Why prompts beat ad-hoc instructions

User types	Uses your prompt
"escalate this"	`/escalate 4821`
Inconsistent results	Consistent every time
No domain expertise	Your expertise baked in
Prompt varies per user	Tested and versioned

In plain English: *The function returns a list of messages that go straight to Claude. You can include multiple back-and-forth turns to guide Claude through complex workflows — like a script it follows every time.*

Users type / in the chat to see available prompt commands. One tap triggers your entire workflow.

2.5 — The MCP Inspector

A browser UI to test your server before writing a single line of client code

Start the inspector:

```
mcp dev mcp_server.py
```

In plain English: *This one command starts your MCP server and opens a browser testing UI. No client code needed — you can try every tool, resource, and prompt interactively.*

Test a full workflow in the inspector:

1. Connect (click the Connect button)
2. Go to **Tools** → List Tools
3. Select `get_ticket_status`
4. Enter `ticket_id = "4821"`
5. Click Run — see the result
6. Now run `add_comment` on the same ticket
7. Verify the comment appears when you re-read the ticket

Inspector = your dev feedback loop

The inspector covers all three server primitives:

Tab	What you test
Tools	List, run with inputs, check outputs
Resources	Browse static + templated resources
Prompts	See exactly what messages go to Claude

Why this matters for PMs:

You can verify exactly what Claude sees and what it can do — without asking an engineer to build a test harness. The inspector is a window into your AI assistant's capabilities.

2.6 — The MCP Client

Two functions — that's all your application needs to implement

Function 1 — Get available tools:

```
async def list_tools(self):
result = await self.session().list_tools()
return result.tools
```

In plain English: *Before sending anything to Claude, your app asks the MCP server: "what tools do you have?" This list gets passed to Claude so it knows what actions it can take.*

Function 2 — Execute a tool:

```
async def call_tool(
self,
tool_name:  str, # e.g.  "get_ticket_status"
tool_input:  dict # e.g.  {"ticket_id":  "4821"}
):
return await self.session().call_tool(
tool_name, tool_input
)
```

Client = the glue between Claude and MCP

Component	Role
ClientSession (SDK)	Manages the actual connection
Your Client class	Wraps session + handles cleanup
`list_tools()`	Discovers tools for Claude
`call_tool()`	Runs what Claude requests

Verify it works:

```
uv run mcp_client.py
# Should print all tool definitions
```

In real projects you build either the client OR the server — not both. We build both

Part 3 — Transport Layer

Message types · Stdio · Streamable HTTP · SSE · Config flags

3.1 — MCP Message Types

All communication is JSON — two categories you need to know

Category	Examples

Request/Result pairs (expect a reply)	`ListToolsRequest` / `ListToolsResult`
	`CallToolRequest` / `CallToolResult`
	`ReadResourceRequest` / `ReadResourceResult`
	`InitializeRequest` / `InitializeResult`

Notifications (one-way, no reply)	Progress Notification Logging Message Notification Tool List Changed Notification Resource Updated Notification

Every connection starts with this 3-message handshake:

```
1. Client:  InitializeRequest
2. Server:  InitializeResult + capabilities
3. Client:  Initialized Notification (no reply needed)
```

MCP is bidirectional — servers can talk back

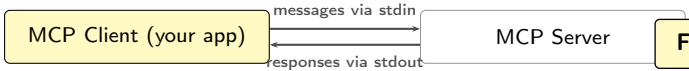
Most protocols are one-way (you ask, server answers). MCP lets the *server* also initiate messages to the client:

- Progress updates during long tasks
- Log messages for debugging
- Asking the client to call Claude (sampling)
- Notifying when data changes

This bidirectionality is powerful — but it creates challenges when deploying over HTTP. That's what the next slides cover.

3.2 — The Stdio Transport

Simplest transport — client and server on the same machine



In plain English: *Stdin/stdout are the simplest data pipes on any computer. The client writes a message to the server's input; the server writes back to its output. Like two people passing notes under a door.*

Test a server directly from terminal:

```
uv run server.py
# Paste a JSON message - see the raw response
```

Four communication patterns

Direction	Channel
Client → Server request	stdin
Server → Client response	stdout
Server → Client request	stdout
Client → Server response	stdin

Characteristics:

- Fully bidirectional — all MCP features work
- Zero network setup
- Same machine only
- Perfect for dev, local tools, desktop apps

3.3 — Streamable HTTP Transport

Remote MCP servers — but HTTP has a fundamental constraint

The HTTP constraint:

HTTP is request-response: the client asks, the server answers. Servers cannot easily *initiate* messages to clients — they don't know the client's address.

Direction	HTTP ease
Client asks server a question	Easy
Server pushes message to client	Hard

In plain English: *Imagine a call center where only customers can call in — agents can't call customers. MCP needs agents (servers) to also be able to reach out. That's the problem SSE solves.*

MCP features that need server → client:

- Sampling (ask client to call Claude)
- Progress notifications during long tasks

The SSE workaround

Server-Sent Events (SSE) = a long-lived HTTP connection the server can write to at any time:

1. Client sends `InitializeRequest`
2. Server replies with a unique `mcp-session-id`
3. Client opens a persistent GET request (SSE channel)
4. Server streams messages through this channel
5. Tool calls get a second, temporary SSE connection

In plain English: *The SSE channel is like leaving the phone line open after saying hello. The server can speak whenever it has something to say, without the client calling again.*

3.4 — Config Flags: `stateless_http` and `json_response`

Two settings that trade features for simplicity — know the cost

Flag	What It Does	What You Lose
`stateless_http=True`	No session IDs. No persistent SSE channel. Server treats every request as brand new.	Sampling, progress reports, notifications, subscriptions, list roots
`json_response=True`	Tool call responses return as plain JSON instead of streaming SSE events.	Intermediate progress messages, log statements during execution

In plain English: *`stateless_http` is like switching from a phone call to email. Every message stands alone — there's no "session" to remember who you're talking to. Simpler, but you lose real-time back-and-forth.*

One benefit of stateless HTTP

Client initialization handshake is skipped. Any request can be made immediately. Critical for load-balanced deployments where you can't guarantee session affinity.

Part 4 — Advanced Topics

Sampling · Logging & Progress · Roots · Scaling · Stateless HTTP

4.1 — Sampling: Letting the Server Ask Claude a Question

The server delegates AI generation to the client — no API key on the server

Scenario: auto-summarise a closed ticket

Server side — request generation:

```
@mcp.tool()
async def summarise_ticket(
ticket_id:  str,
ctx:  Context
):
ticket = zendesk.get(ticket_id)
prompt = f"Summarise this support
ticket in 3 bullet points:
{ticket.full_thread}"
result = await ctx.session.create_message(
messages=[SamplingMessage(
role="user",
content=TextContent(text=prompt)
)],
max_tokens=300,
)
return result.content.text
```

In plain English: *The server fetches the ticket data, builds a prompt, then hands it to the client*

Client side — handle the request:

```
async def sampling_callback(ctx, params):
# Client already has Claude connected
text = await claude.chat(params.messages)
return CreateMessageResult(
role="assistant",
model="claude-sonnet-4-6",
content=TextContent(text=text),
)
```

In plain English: *The client already has an Anthropic API key and an active Claude session. It just passes the server's prompt through its existing connection.*

Why this matters for public chatbots

If you make your MCP server public, you don't want random users generating unlimited Claude output at your cost. Sampling pushes that cost to whoever is

4.2 — Logging & Progress Notifications

Show users what's happening during long-running tasks

Scenario: chatbot triaging 50 tickets overnight

Server side — emit progress:

```
async def triage_all_tickets(
*,
context: Context
):
tickets = zendesk.get_open_tickets()
total = len(tickets) # e.g. 50
for i, ticket in enumerate(tickets):
await context.info(
f"Triaging {ticket.id}..."
)
await context.report_progress(i+1, total)
await triage(ticket)
return f"Done: {total} tickets triaged"
```

In plain English: *After every ticket, the server sends two signals: a log message ("what I'm doing") and a progress update ("how far along I*

Client side — display to user:

```
async def logging_callback(params):
print(f"[LOG] {params.data}")
# [LOG] Triaging ticket 4821...
async def progress_callback(
progress, total, message
):
pct = (progress / total) * 100
print(f"Progress: {pct:.0f}%")
# Progress: 42%
await session.call_tool(
name="triage_all_tickets",
arguments={},
progress_callback=progress_callback
)
```

Web apps: push updates via WebSocket. CLI: print to terminal. Desktop: update a progress bar. The server doesn't care — it just emits.

4.3 — Roots: Granting File System Access

Tell the server which folders it's allowed to touch

Scenario: chatbot that processes uploaded documents

Without roots:

User says "summarise the Q3 report." Claude calls `summarise_file("Q3_report.pdf")`. But the server doesn't know where on the filesystem that file lives — and has no boundaries on what it can access.

With roots:

Claude first calls `list_roots` to see approved folders (e.g. `/uploads/user-123/`), then `read_dir` to find the file, then runs the summary tool with the full path.

You must enforce roots in code

The SDK defines roots but doesn't restrict access automatically. Add this check to every file-touching tool:

```
def is_path_allowed(path):
approved = get_approved_roots()
return any(
path.startswith(root)
for root in approved
)
def summarise_file(file_path:  str):
if not is_path_allowed(file_path):
raise ValueError(
"Access denied:  outside roots"
)
# ...  proceed safely
```

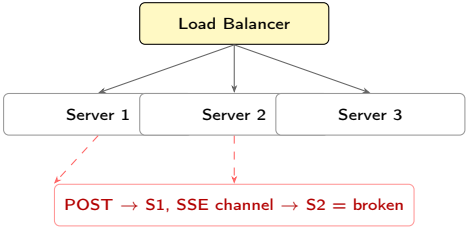
Without this check, roots are advisory only — the server could still read any file on disk.

4.4 — Scaling: Why Stateless HTTP Exists

Horizontal scaling breaks stateful SSE — stateless mode solves it

The scaling problem:

Your support chatbot goes viral. One server can't handle 10,000 simultaneous users. You add a load balancer and three server instances. Now you have a problem:



In plain English: *With a load balancer, one request might hit Server 1 (which holds the SSE session) and the next might hit Server 2 (which knows nothing about that session). The session breaks.*

`stateless_http=True` **removes the coupling**

No sessions → no affinity needed → any server can handle any request.

Stateful	Stateless
Full MCP features	No sampling
Session tracking	No progress updates
Can't load balance	Load balances fine
Single server	Scales horizontally

Use stateless only when scale is the priority and you can live without real-time server-to-client features.

4.5 — Choosing the Right Transport

Match transport to your deployment — don't over-engineer

Transport	Best For	Full MCP Features?	Remote?
Stdio	Dev, local scripts, desktop chatbots	Yes — everything works	No
StreamableHTTP (stateful)	Production chatbots on a single server or controlled infra	Yes — SSE workaround enables all features	Yes
StreamableHTTP (stateless)	High-traffic public bots, microservices, load-balanced deployments	No — no sampling, progress, or notifications	Yes

Golden rule: test with your production transport

Developing with stdio then deploying with stateless HTTP will break sampling, progress bars, and logging. Discover that in dev — not after launch.

Part 5 — Live Session

Building a customer support chatbot MCP server end-to-end.

5.1 — Live Demo Flow

What we build in real-time — support chatbot scenario

1. Create **FastMCP server** with an in-memory ticket store
2. Add **tools**: `get_ticket_status`, `add_comment`, `close_ticket`
3. Add **resources**: list open tickets, fetch ticket by ID
4. Add an **escalation prompt** with priority classification
5. Test everything in the **MCP Inspector**
6. Build the **MCP client** wrapper
7. Wire up the **full end-to-end** chatbot flow
8. Add **progress notifications** to `triage_all_tickets`
9. Add **sampling** to `summarise_ticket`
10. Discuss **transport choice**: when this bot would need stateless HTTP

Every concept from Parts 1–4 appears in this one scenario

5.2 — Prompt Examples: What Good Looks Like

Concrete bad vs. good patterns for AI chatbot workflows

1. Tool description quality

```
Bad:
description="Closes a ticket."
Good:
description="Closes an open support ticket
and sends an automated resolution email
to the customer.  Only call this when the
issue is fully resolved."
```

In plain English: *Claude reads descriptions to decide when to call a tool. Vague descriptions lead to wrong calls — like closing a ticket the user just asked about.*

2. Resource vs Tool

```
Bad:  tool for listing tickets
Good:  resource for listing,
tool only for writing/actions
```

3. Prompt engineering in MCP

```
Bad:
User types:  "escalate ticket 4821"
Claude guesses what escalation means
Good:
@mcp.prompt("escalate") encodes:
- Fetch ticket + full thread
- Classify P1-P4 by severity
- Draft message to engineering
- Suggest next steps for customer
```

4. Sampling vs embedded API key

```
Bad:  API key hardcoded in server
Good:  ctx.session.create_message()
- client pays, server stays clean
```

Part 6 — Expert Principles

The rules that separate good from great MCP implementations.

6.1 — The MCP Cheat Sheet

Decision guide at a glance — for PMs and Applied AI practitioners

Choose the right primitive:

Need	Use
Chatbot takes an action	Tool
Chatbot reads data	Resource
Reusable expert workflow	Prompt
Server needs to call Claude	Sampling
Long task with updates	Progress notifications
File access with guardrails	Roots

Choose the right transport:

Scenario	Transport
Local / dev / desktop	Stdio
Remote, full features	StreamableHTTP stateful
Remote, high traffic	StreamableHTTP stateless

Five principles:

Principle	Why It Matters
Use the SDK	Decorators beat hand-written JSON
Inspector before client	Debug early, not late
Match transport to deploy	Avoid HTTP surprises in prod
Enforce roots in code	SDK won't do it for you
Sampling for public servers	Shift AI cost to the right party

PMs: the primitives (tool/resource/prompt) map directly to product capabilities. Knowing which to use shapes what your AI assistant can do.

6.2 — Seven Principles of MCP Engineering

Carry these into every implementation

1. **Let the SDK do the work.** Decorators and type hints eliminate manual schema writing. Less code = fewer bugs = faster iteration.
2. **Right primitive for the job.** Resources for reads, tools for actions, prompts for expert reusable workflows. Mixing them causes confusion.
3. **Inspector first, client second.** You can verify every tool, resource, and prompt before writing a single line of client code.
4. **Transport choice has real consequences.** Stateless HTTP isn't a safe default. It removes sampling, progress, and notifications — know before you commit.
5. **Enforce your own roots.** The SDK defines roots but won't restrict file access. Add `is_path_allowed()` checks to every file-touching tool.
6. **Sampling protects public chatbots.** Never embed an API key in a public MCP server. Let the client's connection pay for AI generation.
7. **Test with your production transport.** Build and test locally with stdio. Before launch, switch to HTTP and retest — catch transport-specific issues in dev, not in production.

Principle 4 catches most teams off guard

Resources & References

Official Anthropic courses to go deeper after this session

Course 1 — Introduction to MCP

Best for: PMs, Applied AI practitioners, anyone new to MCP

What you'll learn:

- MCP architecture and the request-response flow
- Building servers with tools, resources, and prompts
- Using the MCP Inspector for testing
- Connecting a Python client end-to-end
- When to use each primitive (tools vs resources vs prompts)

Format: Free · Self-paced · Includes assessment

```
anthropic.skilljar.com/
introduction-to-model-context-protocol
```

Course 2 — MCP Advanced Topics

Best for: Engineers and developers building production MCP servers

What you'll learn:

- Sampling — server-to-client AI generation requests
- Progress and logging notifications in real-time
- Roots — file system access control and security
- JSON message architecture (request/result vs notifications)
- Stdio and StreamableHTTP transports in depth
- Stateless HTTP for horizontal scaling with load balancers

Format: Free · Self-paced · Includes assessment

```
anthropic.skilljar.com/
model-context-protocol-advanced-topics
```

Thank You

Vahid Farajijobehdar

Sr. Applied AI Specialist

Applied AI Series — Model Context Protocol (MCP)